

EN UNA FRASE Sabes escribir consultas con `@Query` (JPQL o SQL nativo) cuando Query Derivation se queda corto, y devolver el catálogo en páginas ordenadas con `Pageable`, `Page<T>` y `Sort` desde `GET /libros`.

LO ESENCIAL

JPQL	Lenguaje de consulta de JPA: opera sobre clases y campos Java (<code>FROM Libro l</code>), no sobre tablas y columnas
@Query	Escribe JPQL o SQL nativo en un método del repositorio
@Param("texto")	Vincula un parámetro del método con el marcador <code>:texto</code> de la consulta
nativeQuery = true	El valor de <code>@Query</code> es SQL puro del motor; no es portable entre bases de datos
:param IS NULL OR campo = :param	Filtro opcional: si el parámetro es <code>null</code> , la condición se ignora
@Modifying (con @Transactional)	Obligatorio junto a <code>@Query</code> en un UPDATE o DELETE; el método devuelve las filas afectadas
Pageable	Último parámetro de un método del repositorio: activa paginación y ordenación
PageRequest.of(pagina, tamaño, sort)	Implementación concreta de <code>Pageable</code> ; las páginas empiezan en 0
Page<T>	Datos más metadatos (<code>content</code> , <code>totalElements</code> , <code>totalPages</code> , <code>first</code> , <code>last</code>); ejecuta dos consultas: datos y COUNT
Slice<T>	Versión ligera sin COUNT: solo sabe si hay más (<code>hasNext()</code>); para scroll infinito o «cargar más»
Sort.by("titulo").ascending()	Ordenación dinámica; combina varios campos con <code>Sort.Order.asc(...)</code> y <code>Sort.Order.desc(...)</code>
@RequestParam(defaultValue = "0")	Lee parámetros de la URL (<code>?pagina=0&tamaño=10</code>); con <code>required = false</code> el parámetro es opcional
Proyección de interfaz	Interfaz con getters (<code>ResumenAnual</code>) que Spring Data implementa para recuperar solo unas columnas
PageImpl	Construye un <code>Page</code> a mano en los tests de <code>LibroControllerTest</code>

ERRORES TÍPICOS

1 No property 'campoQueNoExiste' found for type 'Libro'

✓ el cliente ha mandado un `orden` arbitrario. Valídalos contra un `Set.of("id", "titulo", "autor", "anio")` y usa `"titulo"` si no está.

2 ?tamaño=100000 tumba el servidor por memoria

✓ limita el tamaño con `Math.min(tamaño, TAMANIO_MAXIMO_PAGINA)` antes de construir el `Pageable`.

3 La búsqueda con :texto a null falla o no devuelve nada

✓ `CONCAT('%', NULL, '%')` da `NULL`. Comprueba `buscar != null && !buscar.isBlank()` antes de llamar al repositorio.

4 ?pagina=1 devuelve los libros del 11 al 20

✓ las páginas son base 0. Documentalo con `@Parameter(description = "Número de página (base 0)", example = "0")` o resta 1 con `PageRequest.of(Math.max(0, pagina - 1), tamaño, sort)`.

5 SemanticException: entity name used in query [libro] is not defined

✓ has escrito SQL en una `@Query` JPQL. Usa el nombre de la clase y un alias: `SELECT l FROM Libro l`, nunca `SELECT * FROM libro`.

EN EL PROYECTO BIBLIOTECA

`LibroJpaRepository` gana los métodos paginados (`findByTituloIgnoreCase`, `findByAutorIgnoreCase` y `buscarPorTextoPaginado` con `@Query`). `BibliotecaService` añade `obtenerTodosPaginado` y `GET /libros` pasa de devolver `List<Libro>` a `Page<Libro>`, con los parámetros `pagina`, `tamaño`, `orden`, `direccion` y `buscar`. En `LibroControllerTest`, el test del listado se reescribe con `PageImpl` y comprueba `$.content`.

Consultas avanzadas, paginación y ordenación

■ CÓDIGO MÍNIMO Y PETICIONES

Búsqueda por texto paginada en `LibroJpaRepository` (`findAll(Pageable)` ya viene con `JpaRepository`):

```
@Query("""
    SELECT l FROM Libro l
    WHERE LOWER(l.titulo) LIKE LOWER(CONCAT('%', :texto, '%'))
       OR LOWER(l.autor) LIKE LOWER(CONCAT('%', :texto, '%'))
    """)
Page<Libro> buscarPorTextoPaginado(String texto, Pageable pageable);
```

JAVA

El `listarTodos()` de `LibroController`, que ahora devuelve `Page<Libro>`:

```
@GetMapping
public Page<Libro> listarTodos(
    @RequestParam(defaultValue = "0") int pagina,
    @RequestParam(defaultValue = "10") int tamaño,
    @RequestParam(defaultValue = "titulo") String orden,
    @RequestParam(defaultValue = "asc") String direccion,
    @RequestParam(required = false) String buscar) {

    Sort.Direction sentido = "desc".equalsIgnoreCase(direccion)
        ? Sort.Direction.DESC
        : Sort.Direction.ASC;
    Pageable pageable = PageRequest.of(pagina, tamaño, Sort.by(sentido, orden));
    return service.obtenerTodosPaginado(pageable, buscar);
}
```

JAVA

`BibliotecaService` decide entre listar todo o buscar:

```
public Page<Libro> obtenerTodosPaginado(Pageable pageable, String buscar) {
    if (buscar == null || buscar.isBlank()) {
        return repositorio.findAll(pageable);
    }
    return repositorio.buscarPorTextoPaginado(buscar, pageable);
}
```

JAVA

```
### Página 0, 10 libros, ordenados por título (valores por defecto)
GET http://localhost:8080/libros

### Segunda página, 2 resultados por página
GET http://localhost:8080/libros?pagina=1&tamaño=2

### Ordenar por año descendente
GET http://localhost:8080/libros?orden=año&direccion=desc

### Buscar en título o autor, con paginación y orden
GET http://localhost:8080/libros?buscar=code&pagina=0&tamaño=5&orden=año&direccion=asc
```

HTTP